Diurd Vitalii, Oliver Udvardi, CPU, utorok 16:00

Projekt z objektovo orientovaného programovania LS2025

Zámer projektu

My game is a **2D pixel art platformer** that combines the fantasy world of **The Lord of the Rings** with **classic Super Mario-style gameplay**. The player takes on the role of a hero who must navigate a dangerous level filled with traps, enemies, and obstacles to reach the end of the Mines of Moria. Along the way, the player will encounter orcs, which can be avoided or defeated. Mastering timing, dodging, and attacking is essential for survival.

Along the way, the player will encounter **orcs**, which can be **avoided or defeated**. Mastering **timing, dodging, and attacking** is essential for survival.

The **main goal** is to **overcome all obstacles** and **go out of mines of Moria** in an intense battle with enemies. **Winning the game proves the player's skill, courage, and heroism. In addition to the orcs, there are traps scattered throughout the level that must be avoided, adding an extra layer of difficulty. Barrels will also appear, in which players can find healing potions to recover health.**

The game also features a level editor, giving players the ability to draw and create their own levels for even more fun and challenges.

This game is perfect for **fans of platformers and fantasy worlds**, offering **fun and simple gameplay** that is **easy to learn** but still presents a **challenge**. Whether playing casually or aiming for perfection, the game delivers an exciting and rewarding experience that keeps players engaged. **It’s a great mix of action, adventure, and strategy!**

.Žáner: *2d pixelart platformer*

It was decided to make game multi-level without concrete ending, now levels are easymade and allow players to create them by themselves. Also the idea of power-ups was developed and remade to be different types of potions. Traps are other key change in game, they are as dangerous as orcs.

UML diagram tried

Použité knižnice

JUnit for testing game. Mockito to write beautiful tests with a clean & simple API.

Vyjadrenie sa k splneniu podmienok (nutnych, aj dalsich)

Ku kazdej podmienke napiste aspon jeden priklad kde v projekte je splnena, dodrzujte prosim poradie ako je na uvedene na stranke projekt:

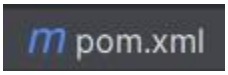
Obsahovat dedenie a rozhrania

```
public class AnimationComponent extends Component {
```

```
public class MovementComponent extends Component {
```

Have inheritance in my code

- V pripade pouzitia inych externych zavislosti musi byt pouzity Maven (napr. jar subory nesmu byt v repozitari)

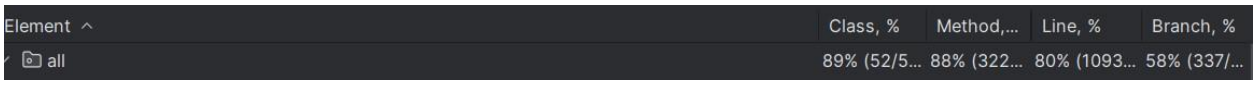


- **Pouzitie zakladnych OOP principov (enkapsulacia, dedenie, polymorfizmus, abstrakcia)**

```
public abstract class Enemy extends Entity {
```

```
public interface IGameObject {
```

- **Musi obsahovat dostatok komentorov a anotacii (JavaDoc)** Competed
- Musi byt jednotkovo otestovany (line coverage > 80% (bezne sa toto blizi k 100%)) – *GUI **nie je** potrebne pre ucely odovzdania projektu jednotkovo otestovat. Gettery a settery **ano**. Toto sa moze pri naozajstnom vyvoji lisit*. V pripade nesplnenia tejto podmienky je stale mozne za projekt ziskat “body”, avsak za tuto cast ziskava (v pripade splnenia ostatnych podmienok) 4b z 8b.



Element	Class, %	Method,...	Line, %	Branch, %
all	89% (52/5...	88% (322...	80% (1093...	58% (337/...

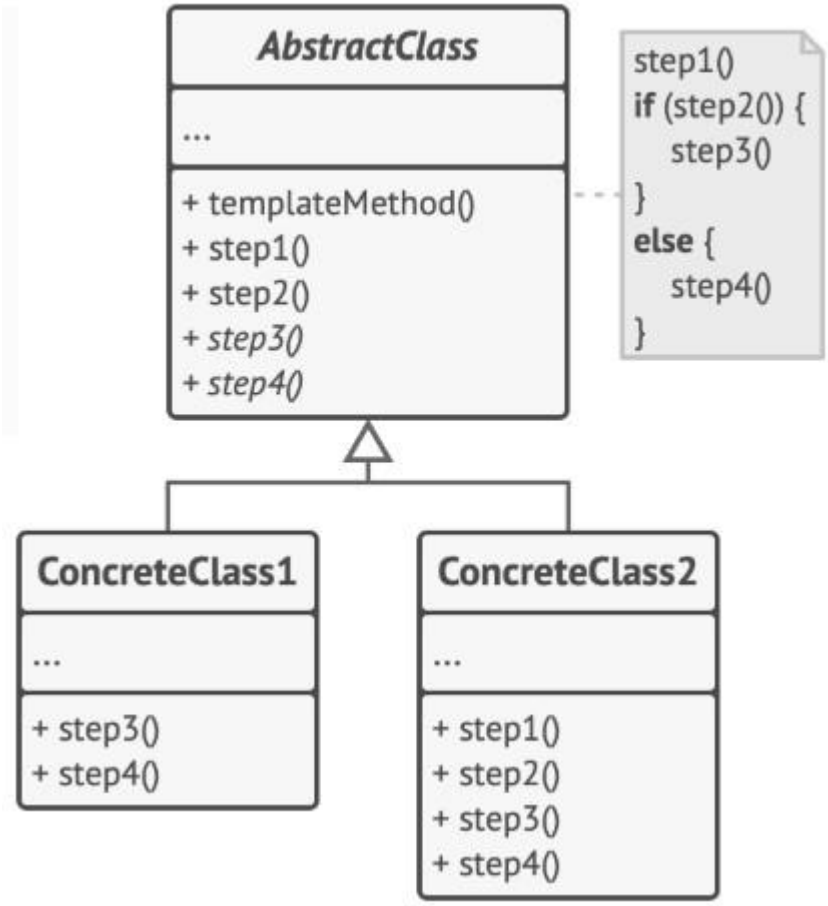
My gui is ui and Inputs

Inputs	100% (4/4)	71% (10/14)	55% (15/27)	27% (3/11)
KeyboardInputs	100% (2/2)	100% (5/5)	80% (8/10)	50% (2/4)
MouseInputs	100% (2/2)	55% (5/9)	41% (7/17)	14% (1/7)

ui	100% (6/6)	76% (45/...	61% (129/...	30% (21/70)

- Musi splnat zakladne principy softveroveho vyvoja, vyvoja objektovo orientovaneho kodu a vyvoja v jazyku Java 21. Completed

- Pouzitie navrhovych/architektonickych vzorov (okrem trivialnych (Singleton)- pouzit ho mozete, ale nebude hodnoteny) – za kazdy vzor 1b (max 3b) 1) Template Pattern



The Template Method pattern suggests that you break down an algorithm into a series of steps, turn these steps into methods, and put a series of calls to these methods inside a single *template method.*

In my case I used abstract class Component

```java
package components;

import entities.Entity;
/**
 * Abstract base class for all components that can be attached to an entity.
 */
public abstract class Component {  5 inheritors  VitaliiDiurd
    protected Entity owner;  21 usages

    /**
     * Sets owner of component.
     * @param owner
     */
    public void setOwner(Entity owner) {  2 overrides  VitaliiDiurd
        this.owner = owner;
    }

    /**
     * Updates the component's logic every game tick.
     */
    public abstract void update();  5 implementations  VitaliiDiurd

}
```

And concrete classes which were responsible for different aspects of player behavior

```java
public class AnimationComponent extends Component {
```

```java
public class InputComponent extends Component {
```

```java
public class MovementComponent extends Component {
```

```java
public class PhysicsComponent extends Component {
```

```java
public class RenderComponent extends Component {
```

Then they are called in Entity

```java
public void addComponent(Component component) {  6 usages  VitaliiDiurd
    component.setOwner(this);
    components.put(component.getClass(), component);
}
```

```java
public <T extends Component> T getComponent(Class<T> componentClass) {  46 usages  VitaliiDiurd
    T component = componentClass.cast(components.get(componentClass));
    if (component == null) {
        throw new IllegalStateException("Component not found: " + componentClass.getSimpleName());
    }
    return component;
}
```

And used in Player class

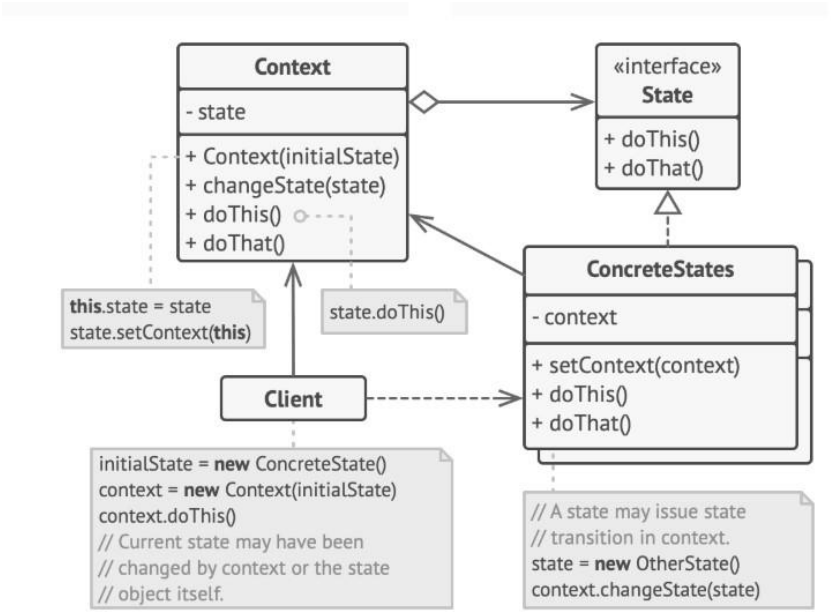
```java
public void loadLvlData(int[][] lvlData) {  4 usages  VitaliiDiurd
    getComponent(PhysicsComponent.class).loadLvlData(lvlData);
    getComponent(MovementComponent.class).loadLvlData(lvlData);
}
```

```java
private void checkAttack() {  1 usage  VitaliiDiurd
    AnimationComponent anim = getComponent(AnimationComponent.class);
    if (anim != null && anim.isAttacking() && anim.getAniIndex() == 1 && !attackChecked) {
        attackChecked = true;
        playing.checkEnemyHit(attackBox);
        playing.checkObhectHit(attackBox);
    }

    if (anim != null && !anim.isAttacking()) {
        attackChecked = false;
    }
}
```

```java
public void render(Graphics g, int lvlOffset) {  1 usage  VitaliiDiu
    getComponent(RenderComponent.class).render(g, lvlOffset);
    drawHitbox(g, lvlOffset);
    drawAttackBox(g, lvlOffset);
}
```

It was made so my Player class was more readable and compact. I believe it really helped me to understand and add new features in my code.

2) State Pattern



The State pattern suggests that you create new classes for all possible states of an object and extract all state-specific behaviors into these classes.

Instead of implementing all behaviors on its own, the original object, called *context*, stores a reference to one of the state objects that represents its current state, and delegates all the state-related work to that object.

In my case I have Context class which is named State

```
public class State {  4 usages  3 inheritors  VitaliiDiurd

    protected Game game;
    public State(Game game) {  3 usages  VitaliiDiurd
        this.game = game;
    }

    /**
     * Checks if the mouse event is within the bounds of th
     * @param e
     * @param mb
     * @return
     */
    public boolean isIn(MouseEvent e, MenuButton mb) {  12 us
        return mb.getBounds().contains(e.getX(), e.getY());
    }

    public Game getGame() {  VitaliiDiurd
        return game;
    }
}
```

And interface StateMethod which provides all methods

```
public interface StateMethods {  2 usages  2 im
    public void update();  2 implementations  V

    public void draw(Graphics g);  2 implementa

    public void mouseClicked(MouseEvent e);

    public void mousePressed(MouseEvent e);

    public void mouseReleased(MouseEvent e);

    public void mouseMoved(MouseEvent e);  7

    public void keyPressed(KeyEvent e);  14 u

    public void keyReleased(KeyEvent e);  10
}
```

Also have enum class which has all States from my game

```
public enum GameState {  VitaliiDiurd
    PLAYING, MENU, OPTIONS, QUIT;  14 usages

    public static GameState state = MENU;

}
```

And finally I have my states

```
public class Playing extends State implements StateMethods {  VitaliiDiurd
    private static final Logger logger = LoggerFactory.getLogger(Playing.class);  46 usage
```

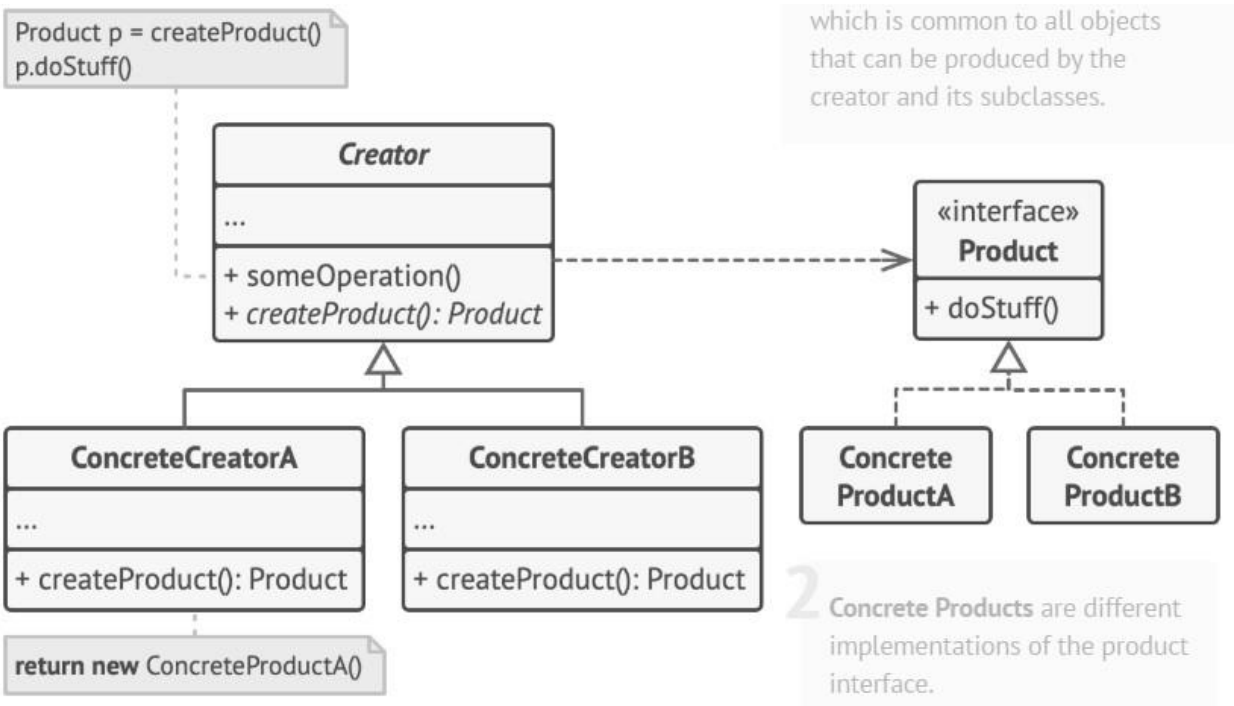
```
public class Menu extends State implements StateMethods {  16
```

And they are called here

```java
public void update(){  VitaliiDiurd

    switch (GameState.state) {
        case MENU:
          menu.update();
          break;
        case PLAYING:
            playing.update();
            break;
        case OPTIONS:
        case QUIT:
        default:
            System.exit( status: 0);
            break;

    }
}
```

It helps me in indicating different sates of my game and adding some new.

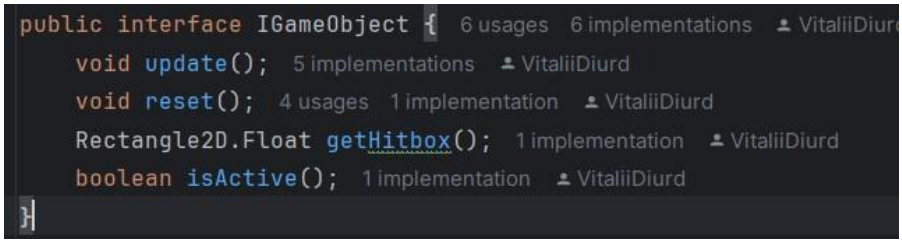
3) Factory Pattern



The Factory Method pattern suggests that you replace direct object construction calls (using the new operator) with calls to a special *factory* method. Don’t worry: the objects are still created

via the new operator, but it's being called from within the factory method. Objects returned by a factory method are often referred to as *products.*

In my case I have interface IGameObject

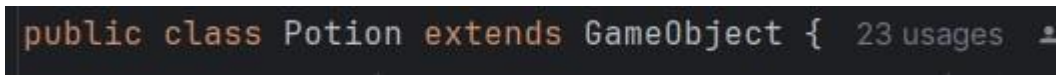


And creator

GameObject Class (Abstract Base Class)

This class implements the IGameObject interface and serves as a base for different types of game objects, such as potions, barrels, traps, etc. It holds common logic that will be shared by all the specific game objects.

And finally I have concrete classes



```
public class GameContainer extends GameObject {
```

Actually I have another implementation of the same pattern

```
public interface EntityFactory {  6 usages  1 implementation  VitaliiDiurd
    Entity createEntity(EntityType type, float x, float y, Playing playing);
}
```

```
public enum EntityType {
    PLAYER,  no usages  2 re
    ORC,  no usages
    BOSS_BALROG  no usages
}
```

```
public class DefaultEntityFactory implements EntityFactory {  7 usages  VitaliiDiurd

    public Entity createEntity(EntityType type, float x, float y, Playing playing) {  1 usage
        return switch (type) {
            case PLAYER -> createPlayer(x, y, playing);
            default -> throw new IllegalArgumentException("Unsupported entity type: " + type);
        };
    }
```

```
private Player createPlayer(float x, float y, Playing playing) {  1 usage  VitaliiDiurd
    Player player = new Player(x, y,  width: 32,  height: 32, playing);
    try {
        player.addComponent(new AnimationComponent( atlasPath: "/gandalf_atlas.png"));
    } catch (ResourceLoadException e) {
        throw new RuntimeException("Failed to load player animation", e);
    }
    player.addComponent(new PhysicsComponent());
    player.addComponent(new MovementComponent());
    player.addComponent(new InputComponent());
    player.addComponent(new RenderComponent());
    return player;
}
```

- Logovanie zakladnych cinnosti (aspon na urovni info, warning, error) – 1b

I have loggers in class Playing in package gamestates

```
public class Playing extends State implements StateMethods {  VitaliiDiurd
    private static final Logger logger = LoggerFactory.getLogger(Playing.class);  46 usages
```

```
try {
    initClasses();
    backgroundImg = LoadSave.GetSpriteAtlas(LoadSave.GAME_BACKGROUND);
    logger.debug("Successfully loaded background image");

    calcLvlOffset();
    loadStartLevel();
    logger.info("Playing state initialized successfully");
} catch (Exception e) {
    logger.error("Failed to initialize Playing state", e);
    throw new RuntimeException("Playing initialization failed", e);
}
```

```
logger.debug("Loading start level data");
try {
    enemyManager.loadEnemies(levelManager.getCurrentLevel());
    objectManager.loadObjects(levelManager.getCurrentLevel());
} catch (Exception e) {
    logger.error("Failed to load start level data", e);
    throw new RuntimeException("Level data loading failed", e);
```

```
logger.info("Created player entity at (200,200)");
```

INFO

```
logger.debug("Initialized all UI overlays");
```

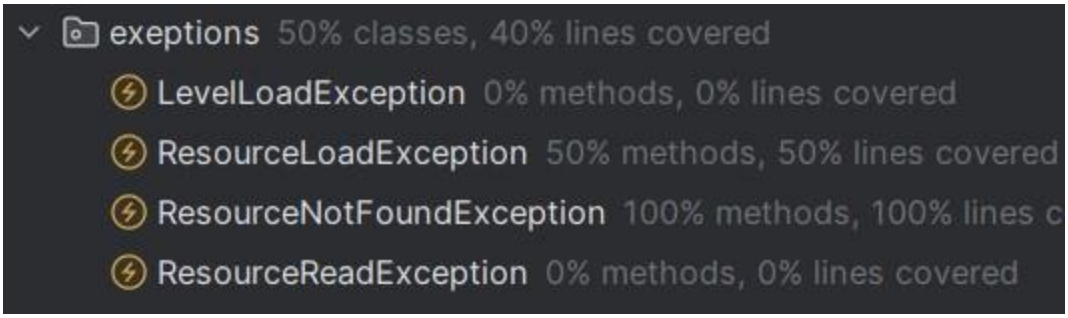
WARNING

```
logger.error("Failed to initialize game components", e);
```

-

ERROR

Implementacia a pouzitie vlastnych vynimiek



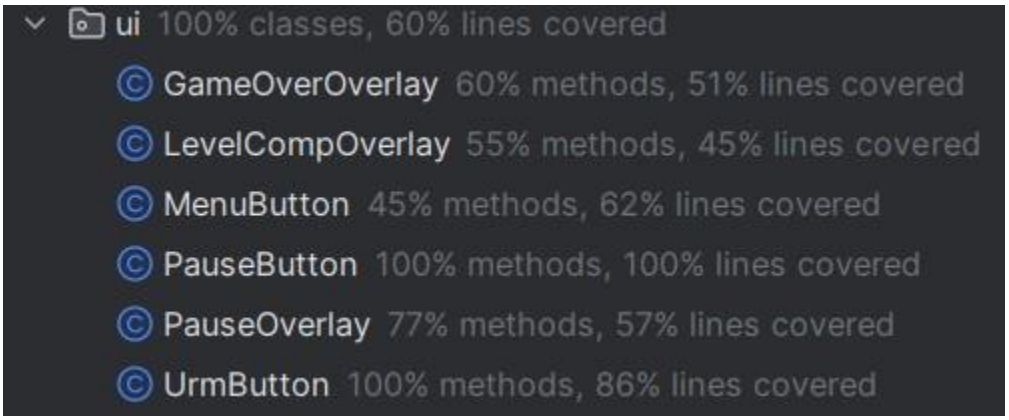

```
private void loadAnimations() throws ResourceLoadException {
```

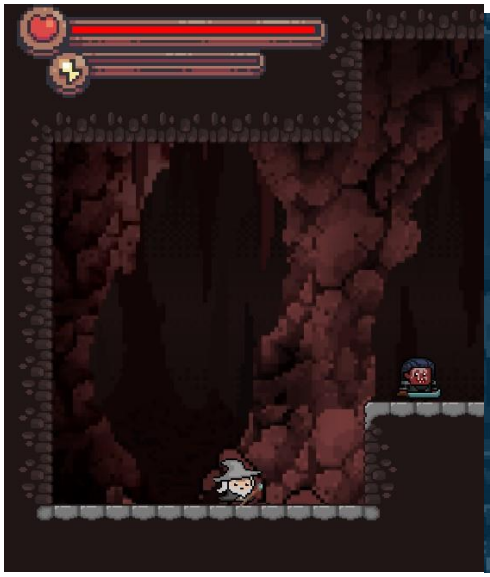
```
throw new ResourceLoadException("Failed to load animation frame at (" + i + ", " + j + ")");
```

```
} catch (ResourceNotFoundException e) {
    System.err.println("Failed to load orc enemy images: " + e.getMessage());
```

```
throw new ResourceReadException(fileName, new IOException("ImageIO r
```

- Implementacia a vytvorenie GUI – 2b





Explicitne pouzitie viacvlaknovosti

```
private void startGameLoop() {  1 usage   VitaliiDiurd
    gameThread = new Thread( task: this);
    gameThread.start();
}
```

Using it in Game class to implement GameLoop

Pouzitie generickovsti vo vlasnych triedach

```
public <T extends Component> T getComponent(Class<T> componentClass) {  46 usages   VitaliiDiurd
    T component = componentClass.cast(components.get(componentClass));
    if (component == null) {
        throw new IllegalStateException("Component not found: " + componentClass.getSimpleName())
    }
    return component;
}
```

Used in Entity class

The use of generics (<T extends Component>) in this method allows it to safely return a specific type of component without needing to manually cast it later. It makes the code more type-safe, reusable, and avoids potential ClassCastException errors at runtime.

Pouzitie reflexie

```
var buttonsField = Menu.class.getDeclaredField( name: "buttons");
buttonsField.setAccessible(true);
buttonsField.set(menu, mockButtons);

var backgroundField = Menu.class.getDeclaredField( name: "backgroundImg");
backgroundField.setAccessible(true);
backgroundField.set(menu, mockBackgroundImg);

var menuImgField = Menu.class.getDeclaredField( name: "menuImg");
menuImgField.setAccessible(true);
menuImgField.set(menu, mockMenuImg);
```

In the MenuTest class, reflection is used to access and modify private fields of the Menu class in order to inject mock objects. This is achieved through the methods getDeclaredField, setAccessible(true)

```
method.setAccessible(true);
```

```
field.setAccessible(true);
```

```
lvlIndexField.setAccessible(true);
```

Pouzitie lambda vyrazov, referencii na metody

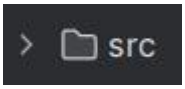
```
public void checkObjectTouched(Rectangle2D.Float hitbox) {  1 usage   VitaliiDiurd
    if (potions == null) return;
    potions.stream() Stream<Potion>
            .filter( Potion p -> p.isActive() && hitbox.intersects(p.getHitbox()))
            .findFirst() Optional<Potion>
            .ifPresent( Potion p -> {
                p.setActive(false);
                applyEffect(p);
            });
}
```

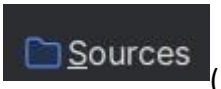
```
public void update() {   VitaliiDiurd
    if (potions != null)
        potions.stream()
                .filter(Potion::isActive)
                .forEach(Potion::update);

    if (containers != null)
        containers.stream()
                .filter(GameContainer::isActive)
                .forEach(GameContainer::update);
}
```

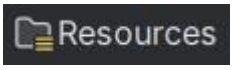
Navod ako spustit projekt

1) Download zip from my repository
2) Move it to created previously folder
3) Open folder, click on project-VitaliiDiurd-main with right button and open Folder with IntelijIdea
4) Go to “burger” in the left top

5) Then press Project Structure 

6) Then modules, click on 

and 

(now package should be blue)
7) Open src go to main, right click on res and 

8) Now in project go src -> main -> MainClass and run

Pouzivatelska prirucka

Click left or right to move, space to jump right click to attack. You can destroy barrels and gain health using blue and red potion. Avoid traps because they are killing instantly.To go to other level you need to kill all enemies from present level.